ASSIGNMENT – 5

Introduction to Hash Cryptography

Subject: Cryptography and Network Security

Submitted by: Shreya Paliwal

Course: BCA

Q1. Define hash cryptography. Explain the fundamental principles and techniques of cryptographic hash functions with suitable examples.

Hash cryptography is a technique that uses a mathematical function called a cryptographic hash function to convert data of any length into a fixed-length value called a hash, digest, or message digest. A hash is designed to represent the original data uniquely in practical use, although different inputs can theoretically produce the same hash.

Fundamental principles:
1. Deterministic: The same input always produces the same hash.
2. Fixed-length output: The output length is fixed for a particular algorithm, regardless of input size. For example, SHA-256 produces a 256-bit digest.
3. One-way operation: It should be computationally infeasible to recover the original input from its hash.
4. Avalanche effect: A small change in the input should cause a large, unpredictable change in the output.
5. Collision resistance: It should be computationally infeasible to find two different inputs with the same hash.

Techniques and working:
• The input message is processed in blocks or through a defined internal state.
• Padding and length encoding may be applied according to the algorithm.
• Repeated rounds of bitwise operations, modular arithmetic, permutations, or other transformations mix the input data.
• The final internal state is returned as the fixed-length digest.

Example: A file is processed with SHA-256 to produce a digest. The receiver calculates the SHA-256 digest of the downloaded file and compares it with a trusted published digest. If the values match, this provides evidence that the file has not changed, assuming the published digest is authentic and the algorithm remains secure.

A hash function is not encryption: it does not use a decryption key to recover the original message. It is mainly used for integrity checks, digital signatures, and other security mechanisms.

Q2. Discuss the role of cryptographic hash functions in cryptographic applications. Explain their use in data integrity verification, digital signatures, password storage, and message authentication.

Cryptographic hash functions are basic building blocks in information security. They create fixed-length digests that can be used to check data, support signatures, and form part of authentication systems.

1. Data integrity verification:
A sender or publisher provides a trusted hash digest for a file or message. The receiver hashes the received data and compares the result. A match suggests the content has not changed. The reference digest must itself come from a trusted source; otherwise, an attacker could replace both the file and digest.

2. Digital signatures:
Instead of signing a large document directly, a signing system normally hashes the document and signs the digest with the sender's private key. The recipient verifies the signature using the corresponding public key and checks the digest. This helps detect changes and supports authentication and non-repudiation when the complete signature system and key management are trustworthy.

3. Password storage:
Systems should not store plain-text passwords. They should store a password verifier generated using a password-hashing scheme. Modern practice uses a unique random salt and a deliberately slow password-hashing algorithm such as Argon2id, scrypt, or bcrypt; PBKDF2 is also used. A fast general-purpose hash such as plain SHA-256 alone is not recommended for password storage because attackers can test guesses quickly.

4. Message authentication:
A Message Authentication Code (MAC) helps verify message integrity and authenticity using a shared secret. HMAC is a widely used construction that combines a cryptographic hash function, such as SHA-256, with a secret key. A plain hash alone does not prove who sent a message because anyone can calculate it.

Thus, hash functions help make many security operations efficient, but they must be used in the correct construction and with secure key management.

Q3. Explain the advantages and disadvantages of hash cryptography. Discuss the major properties of a secure cryptographic hash function, including preimage resistance, second-preimage resistance, and collision resistance.

Advantages:
1. Fast and efficient: Hashes can be calculated for files and messages of many sizes.
2. Fixed-size digest: A large message is represented by a compact value.
3. Integrity checking: Comparing digests can reveal accidental or malicious changes when the reference digest is trusted.
4. Supports other security tools: Hashes are used in digital signatures, HMAC, certificates, and secure protocols.
5. No decryption key is needed: A hash is designed as a one-way operation rather than reversible encryption.

Disadvantages and limitations:
1. Hashing alone does not provide confidentiality because it does not hide data in the way encryption does.
2. A plain hash does not authenticate the sender; an attacker may calculate a new hash for altered data.
3. Collisions are mathematically possible because there are more possible inputs than fixed-length outputs.
4. Fast hashes are unsuitable by themselves for password storage.
5. Weak or outdated algorithms can allow practical attacks. For example, SHA-1 is no longer recommended where collision resistance is required.

Major security properties:
• Preimage resistance: Given a hash value h, it should be computationally infeasible to find any input m such that H(m) = h.
• Second-preimage resistance: Given a specific input m1, it should be infeasible to find a different input m2 for which H(m1) = H(m2).
• Collision resistance: It should be infeasible to find any two distinct inputs m1 and m2 such that H(m1) = H(m2). This property is important for signatures and certificates.

These properties are related but different. A secure algorithm should make the relevant attacks computationally impractical, not mathematically impossible.

Q4. Describe the Secure Hash Algorithm (SHA) family. Explain the development and key features of SHA-1, SHA-2, and SHA-3, highlighting their major differences.

The Secure Hash Algorithm (SHA) family is a set of cryptographic hash standards developed by the U.S. National Institute of Standards and Technology (NIST). The main groups commonly discussed are SHA-1, SHA-2, and SHA-3.

1. SHA-1:
• Produces a 160-bit (20-byte) digest.
• Uses a 512-bit message block and 80 processing rounds.
• It was widely used in older digital signatures and integrity systems.
• Practical collision attacks have made SHA-1 unsuitable for security applications that depend on collision resistance. New systems should not use it for digital signatures or similar purposes.

2. SHA-2:
• Includes SHA-224, SHA-256, SHA-384, and SHA-512, along with additional variants.
• SHA-256 produces a 256-bit digest; SHA-512 produces a 512-bit digest.
• It is widely used in modern protocols, certificates, digital signatures, and integrity checking.
• SHA-2 remains a recommended family when implemented correctly and used in an appropriate protocol.

3. SHA-3:
• Standardized by NIST in 2015 and based on the Keccak design.
• Includes SHA3-224, SHA3-256, SHA3-384, and SHA3-512.

• Uses a sponge construction: data is absorbed into an internal state and the digest is then squeezed out.
• Its internal design differs from SHA-1 and SHA-2, providing an alternative construction. SHA-3 is useful where a different hash design is desired.

Major differences:
• Digest size: SHA-1 gives 160 bits; SHA-2 and SHA-3 offer several digest sizes, including 256 and 512 bits.
• Internal design: SHA-1 and SHA-2 use related block-processing designs, while SHA-3 uses the Keccak sponge construction.
• Security status: SHA-1 is deprecated for collision-sensitive security uses; SHA-2 and SHA-3 are considered secure when used correctly.
• Compatibility: SHA-2 has broad deployment in existing systems, while SHA-3 provides a complementary alternative.

The algorithm should be selected according to current standards, protocol requirements, and compatibility needs.

Q5. Explain the applications of SHA algorithms in cryptographic protocols and information security. Discuss their role in digital certificates, digital signatures, file integrity checking, and secure communication.

Secure Hash Algorithm (SHA) functions are used to create message digests that support integrity and authentication mechanisms in many information-security systems.

1. Digital certificates:
Certificates bind an identity to a public key and are digitally signed by a certificate authority. SHA-256 or another approved hash may be used as part of the signature algorithm. A verifier checks the certificate's signature and validates the certificate chain, validity period, identity, and trust status. A hash alone does not make a certificate trustworthy.

2. Digital signatures:
The message or document is hashed, and the resulting digest is processed by the digital-signature algorithm with the signer's private key. The recipient verifies the signature with the public key and checks that the message digest matches. This helps detect document changes and supports origin authentication when the key is properly linked to the signer.

3. File integrity checking:
Software publishers may publish SHA-256 checksums for installation files. A user calculates the digest of the downloaded file and compares it with the value obtained from an authentic source. Matching values provide evidence that the file has not been modified. If an attacker can replace both the file and checksum, the comparison is not sufficient.

4. Secure communication:
Protocols such as TLS use hash functions in handshake transcript processing, digital-signature schemes, and key-derivation or authentication constructions, depending on the protocol version and cipher suite. HMAC-based constructions may use SHA-256 to authenticate messages. Hashes contribute to integrity and authentication, while encryption provides confidentiality.

5. Other applications:
SHA-based hashes are used in software distribution, version-control systems, forensic verification, and data-management systems. The algorithm and protocol must be chosen carefully, and secret keys must be protected where authentication depends on them.

In conclusion, SHA algorithms are important components of modern cryptography, but security depends on the complete protocol, implementation, trusted keys, and use of current algorithms.